ЯЗИМЕП II

1. Общие положения
В рамках дисциплины предлагается выполнить **5 комплексных проектных заданий**, каждое из которых представляет собой самостоятельное веб-приложение или веб-сервис.
Задания направлены на развитие следующих компетенций:

- проектирование программных систем;
- применение объектно-ориентированного программирования (ООП);
- разработка веб-API;
- работа с данными и базами данных;
- реализация бизнес-логики;
- выбор и обоснование архитектурных решений.

2. Используемые языки программирования
В рамках выполнения заданий используются следующие языки:

- **Python** — обязательно
- **Go** — обязательно
- **C# ИЛИ Java** — на выбор
- **JavaScript ИЛИ TypeScript** — на выбор

3. Общие требования к каждому проекту
Каждое задание должно включать:
3.1. Серверная часть

- реализация HTTP API;
- обработка ошибок;
- валидация входных данных;
- логирование.

3.2. Работа с данными

- использование базы данных (SQL или NoSQL);
- корректная работа с сущностями и связями.

3.3. Архитектура

- разделение на слои:
  - контроллеры (API);
  - сервисы (бизнес-логика);
  - репозитории (доступ к данным);
- использование DTO для передачи данных;
- осмысленное применение (или осознанный отказ от) ООП.

3.4. Документация

- README с инструкцией запуска;
- описание API (допускается использование Swagger/OpenAPI).

3.5. Тестирование (минимально)

- наличие хотя бы базовых тестов ключевой логики.

Задание 1
Сервис бронирования ресурсов

Язык: C# **ИЛИ** Java

Тип: Объектно-ориентированная архитектура (обязательно)

Цель
Разработать веб-приложение для бронирования ресурсов (переговорные комнаты, оборудование и т.п.).

Функциональные требования
Система должна обеспечивать:
- регистрацию и аутентификацию пользователей;
- управление ресурсами (создание, просмотр);
- создание бронирований;
- изменение и отмену бронирований;
- проверку пересечений по времени;
- ограничения (например: не более N бронирований в день).

Архитектурные требования
Обязательно:
- выделение **доменных сущностей** (User, Resource, Booking и др.);
- использование слоёв:
  - Controller
  - Service
  - Repository
- использование DTO;
- применение принципов ООП:
  - инкапсуляция;
  - абстракции;
  - (при необходимости) наследование.

Дополнительные усложнения
- роли пользователей (администратор/обычный пользователь);
- правила доступа к ресурсам;
- поддержка повторяющихся бронирований;
- уведомления (логические, без реальной отправки).

Задание 2
Сервис сокращения ссылок (URL Shortener)

Язык: Go

Тип: Системное проектирование, конкурентность

Цель
Разработать сервис для создания коротких ссылок и сбора статистики переходов.

Функциональные требования
- создание короткой ссылки;
- редирект по короткому URL;
- сбор статистики:
  - количество переходов;
  - время;
  - user-agent;
- API для получения аналитики.

Архитектурные требования
- реализация HTTP-сервера;
- использование middleware (логирование, обработка ошибок);
- потокобезопасная работа со счётчиками;
- модульная структура проекта.

Особое требование
Задание **не требует обязательного применения ООП**.
Необходимо продемонстрировать грамотную архитектуру с использованием:
- интерфейсов;
- композиции;
- функций и структур.

Дополнительные усложнения
- rate limiting;
- кэширование популярных ссылок;
- срок жизни ссылки;
- фоновые задачи (очистка устаревших данных).

Задание 3
Система модерации пользовательского контента

Язык: Python

Тип: Сравнение подходов (ООП vs процедурный)

Цель
Разработать сервис автоматической модерации текстового контента.

Функциональные требования
- API для отправки текста;
- проверка текста по правилам:
  - запрещённые слова;
  - повторения;
  - длина текста;
  - наличие ссылок;
- результат проверки:
  - approved / rejected / manual_review;
- хранение истории проверок.

Ключевое требование
Реализовать **два варианта модуля проверки**:
Вариант 1 — без ООП
- функции;
- пайплайн обработки;
- простые структуры данных.

Вариант 2 — с ООП
- базовый класс/интерфейс Rule;
- отдельные классы правил;
- расширяемая архитектура.

Предоставить сравнительный анализ:
- удобство расширения;
- читаемость;
- тестируемость;
- применимость подходов.

Дополнительные усложнения
- настраиваемые правила;
- приоритеты правил;
- панель администратора.

Задание 4
Интернет-магазин с системой скидок

Язык: JavaScript **ИЛИ** TypeScript

Тип: Сложная бизнес-логика

Цель
Разработать backend интернет-магазина с поддержкой гибкой системы скидок.

Функциональные требования
- каталог товаров;
- корзина;
- оформление заказа;
- история заказов.

Ключевая часть
Реализовать **движок расчёта стоимости заказа**, поддерживающий:
- процентные скидки;
- фиксированные скидки;
- скидки по категориям;
- промокоды;
- комбинирование правил.

Архитектурные требования
- модульность;
- разделение логики расчёта;
- валидация входных данных;
- API.

Дополнительные усложнения
- приоритеты скидок;
- объяснение расчёта цены;
- конфликты правил;
- кэширование результатов.

Задание 5
Платформа онлайн-тестирования

Язык: C# **ИЛИ** Java

Тип: Объектная модель + архитектура

Цель
Разработать систему для создания и прохождения тестов.

Функциональные требования
- роли: преподаватель / студент;
- создание тестов;
- разные типы вопросов:
  - одиночный выбор;
  - множественный выбор;
  - текстовый ответ;
- прохождение теста;
- автоматическая проверка;
- хранение результатов.

Архитектурные требования
- объектная модель:
  - Question (базовый класс);
  - наследники типов вопросов;
- разделение логики оценки;
- использование DTO;
- многослойная архитектура.

Дополнительные усложнения
- случайная генерация вариантов теста;
- ограничение по времени;
- защита от повторной отправки;
- аналитика результатов.